Pointer

11 April 2026 20:03

What is a Pointer?

A pointer is a variable that stores the memory address of another variable.

Instead of storing value, it stores the location where the value is stored.

Basic syntax:

data_type *pointer_name;

For example:

int *p;

- p is a pointer to integer
- It can store the address of an integer variable.

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int a = 10;
    int *p;
    p = &a; // Stores address of a in pointer p
    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    printf("Value stored in p (address of a) = %p\n", p);
    printf("Value at address stored in p = %d\n", *p);
    printf("Address of pointer p = %p\n", &p);
    return 0;
}
```

& ← Address Operator

* ← Dereference Operator. (it is used to access the data of pointer as an address of another variable. Using this address it fetches data present on that address).

Output:

```
Value of a = 10
Address of a = 00000000005ffecc
Value stored in p (address of a) = 00000000005ffecc
Value at address stored in p = 10
Address of pointer p = 00000000005ffec0
```

int a $\boxed{10}$ 5ffec4

int *p $\boxed{5ffec4}$ 5ffec0

Types of Pointers

1. NULL pointer

int *p = NULL;

- **Points to nothing**
- **Good practice to initialize pointers.**

2. void Pointer

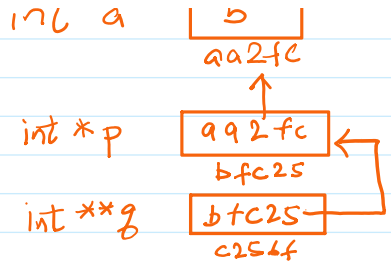
void *p;

- **Can point to any data type.**
- **Needs typecasting to dereference.**

int a $\boxed{5}$ aa2fc

3. Pointer to Pointer

- Needs typecasting to dereference.



3. Pointer to Pointer

```
int a = 5;
int *p = &a;
int **q = &p;
```

- q stores address of pointer p

4. Wild pointer

```
int *p; //uninitialized pointer
```

- Dangerous: contains garbage address.

5. Dangling Pointer

```
int *p;
{
   int x = 10;
   p = &x;
} //x goes out of scope

//p is now dangling
```

6. Pointer Arithmetic: Pointers can be incremented or decremented.

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int ar[] = {1, 2, 3};
    int *p = ar;
    printf("%d\n", *p); //1
    p++; //pointer will be updated to next address
    printf("%d\n", *p); //2
    p++;
    printf("%d\n", *p); //3
    p--;
    printf("%d\n", *p); //2
    return 0;
}
```

Output:

```
1
2
3
2
```

8. Pointer with Arrays

```c
#include<stdio.h>
#define SIZE 3
int main(int argc, char const *argv[])
{
    int ar[] = {1, 2, 3};
    int i;
```

```
    for(i = 0; i < SIZE; i++){
        printf("%d\n",*(ar + i));
    }

    return 0;
}
```

Output:

```
1
2
3
```

Array name acts as a pointer to the first element.

9. Pointer with functions(Call by reference)

```
#include<stdio.h>
void swap(int *x, int *y){
    int t = *x;
    *x = *y;
    *y = t;
}
int main(int argc, char const *argv[])
{
    int a = 5, b = 10;
    printf("Values before swapping a = %d, b = %d\n", a,b);
    swap(&a, &b);
    printf("Values after swapping a = %d, b = %d\n", a,b);
    return 0;
}
```

Output:

```
Values before swapping a = 5, b = 10
Values after swapping a = 10, b = 5
```

- **Allows modifying original variables.**

Heap memory area, is very large memory, it is used to allocate memory during execution of code (dynamic memory allocation).

Dynamic memory allocation

In C malloc() and calloc() are used for dynamic memory allocation - allocating memory at runtime from the heap using function defined in C <stdlib.h>.

malloc(): (memory allocation) Allocates a single block of memory of specified size.

```
ptr = (type *)malloc(size_in_bytes);
```

Example code:

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char const *argv[])
```

```
{
    int n, i, *ptr;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    ptr = (int *) malloc(sizeof(int)*n);
    if(ptr == NULL){
        printf("Memory not allocated!");
        return -1;
    }
    //Memory allocated assign values to it
    for(i = 0; i < n ; i++){
        ptr[i] = (i+1) * 10 ;
    }
    //printing content of array
    printf("content of ptr[]:\n");
    for(i = 0; i < n ; i++){
        printf("%d\n", ptr[i]);
    }

    free(ptr); //releases the memory
    return 0;
}
```

Output:

```
Enter number of elements: 6
content of ptr[]:
10
20
30
40
50
60
```

- **Important points about malloc()**
    - **Allocates uninitialized memory (contains garbage values).**
    - **Returns void pointer (void *)**
    - **Must be typecasted (Optional in modern C, but commonly done in teaching)**
    - **Returns NULL if allocation fails.**

2. calloc() - Contiguous Allocation

It allocate memory for multiple elements and initializes all bytes to zero if it is integer types, '\0' if it is char, 0.0f for floating point, and 0.0 for double type, for struct type it is NULL.

```
ptr = (type *) calloc(n, size);
```

- n -> number of elements
- size -> size of each element

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char const *argv[])
{
    int n, i, *ptr;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);
    //ptr = (int *) malloc(sizeof(int)*n);
    ptr = (int *) calloc(n, sizeof(int));
    if(ptr == NULL){
        printf("Memory not allocated!");
        return -1;
    }
    //Memory allocated assign values to it
    for(i = 0; i < n ; i++){
        ptr[i] = (i+1) * 10 ;
    }
    //printing content of array
    printf("content of ptr[]:\n");
    for(i = 0; i < n ; i++){
        printf("%d\n", ptr[i]);
    }

    free(ptr); //releases the memory
    return 0;
}
```

```
Output:
   Enter number of elements: 6
   content of ptr[]:
   10
   20
   30
   40
   50
   60
```

Important points about calloc()

- **Allocates multiple blocks of memory.**
- **Slightly slower than malloc() due to initialization.**

Code to differentiate malloc() and calloc()

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char const *argv[])
{
    int *m, *c;
    m = (int *)malloc(5 * sizeof(int));
    c = (int *)calloc(5, sizeof(int));
    printf("malloc values: \n");
    int i;
    for(i = 0; i < 5; i++){
        printf("%d\n", m[i]); //garbage
    }

    printf("calloc values: \n");
    for(i = 0; i < 5; i++){
        printf("%d\n", c[i]); //0
    }
```

```
    free(m);
    free(c);

    return 0;
}
```

Output:

```
malloc values:
-1163005939
-1163005939
-1163005939
-1163005939
-1163005939
calloc values:
0
0
0
0
0
```

realloc() -> Reallocation of memory: Resize the previously allocated memory (from malloc() or calloc())

```
ptr = (type *) realloc(ptr, new_size);
```

- **ptr-> previously allocated memory**
- **new_size -> new size in bytes**

What it does?

- **Expands or shrinks memory block.**
- **May move to a new location.**
- **Preserves old data (up to min (old, new))**
- **It returns null if resizing fails.**

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char const *argv[])
{
    int n, i, *ptr;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    //ptr = (int *) malloc(sizeof(int)*n);
    ptr = (int *) calloc(n, sizeof(int));
    if(ptr == NULL){
        printf("Memory not allocated!");
        return -1;
    }
    //Memory allocated assign values to it
    for(i = 0; i < n ; i++){
        ptr[i] = (i+1) * 10 ;
    }
    //printing content of array
    printf("content of ptr[]:\n");
```

```c
    for(i = 0; i < n ; i++){
        printf("%d\n", ptr[i]);
    }

    ptr = (int *) realloc(ptr, 10 * sizeof(int));
    if(ptr == NULL){ //safe coding
        printf("Memory not re-allocated!");
        return -1;
    }
    for(i = n; i < 10; i++){
        ptr[i] = (i+1) * 10 ;
    }
    //printing content of extended array
    printf("content of extended ptr[]:\n");
    for(i = 0; i < 10; i++){
        printf("%d\n", ptr[i]);
    }
    free(ptr); //releases the memory
    return 0;
}
```

← can shrink or extend the memory location

Output:

```
Enter number of elements: 5
content of ptr[]:
10
20
30
40
50
content of extended ptr[]:
10
20
30
40
50
60
70
80
90
100
```